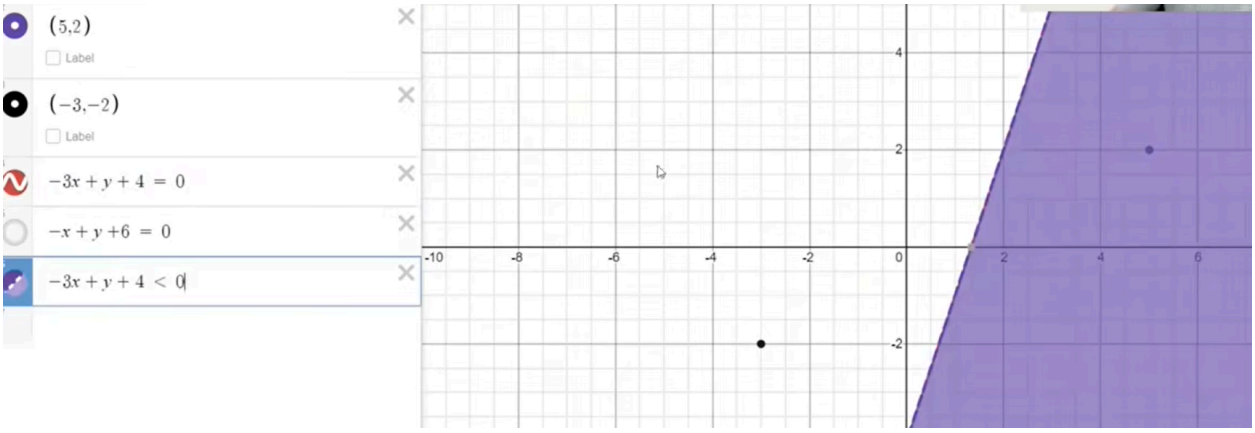
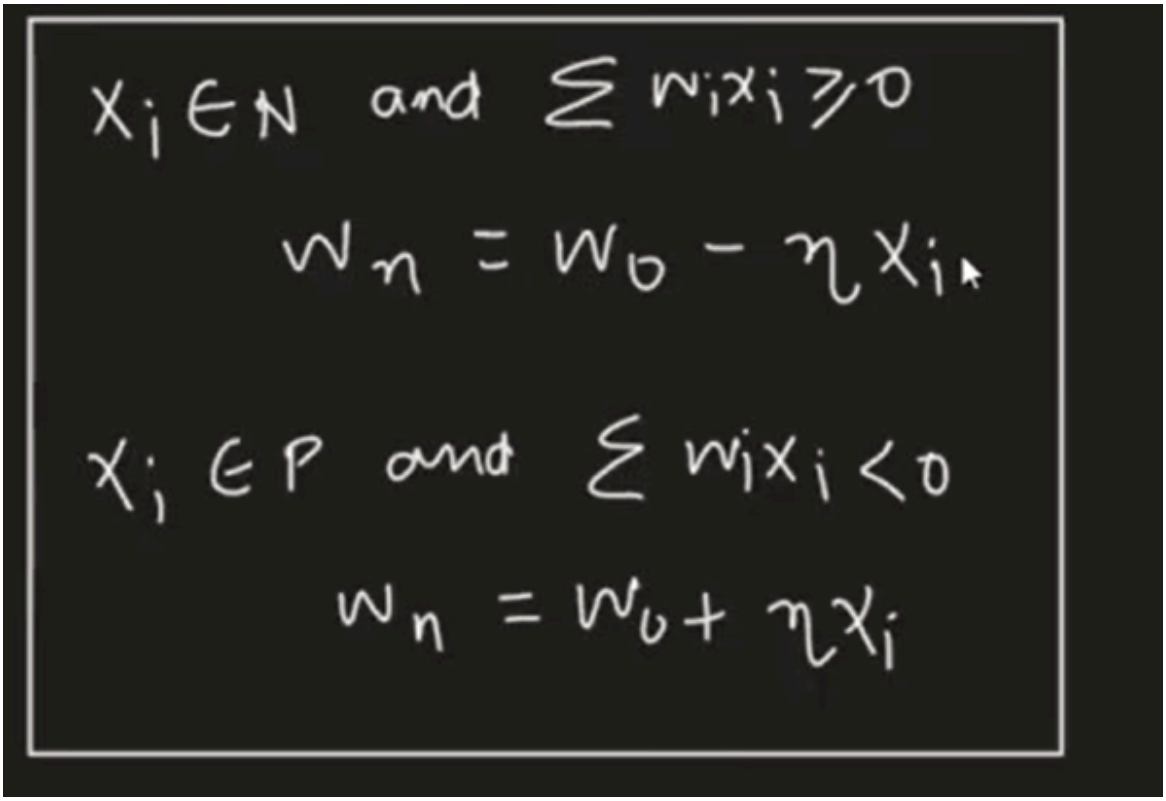
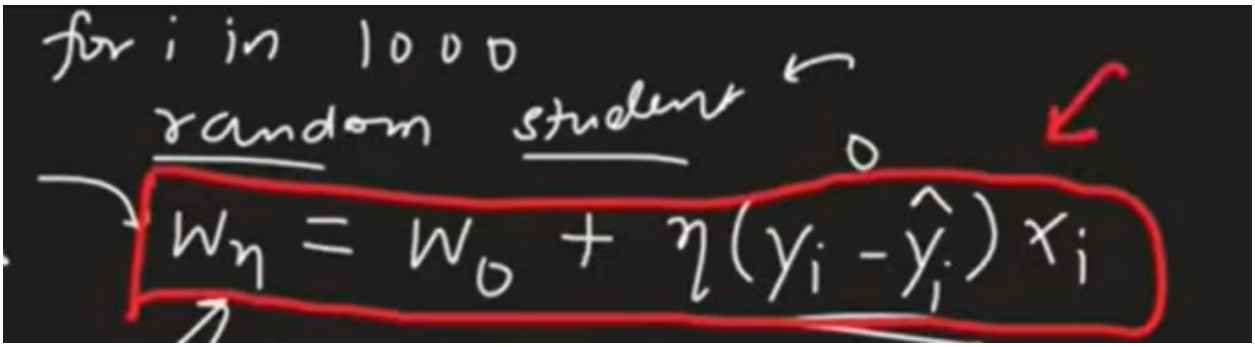
MLP

perceptron:

if a point belongs to negative but is in positive region, then weights should be minused using the point's coordinates and bias should be chosen as well , Happens the opposite vice versa





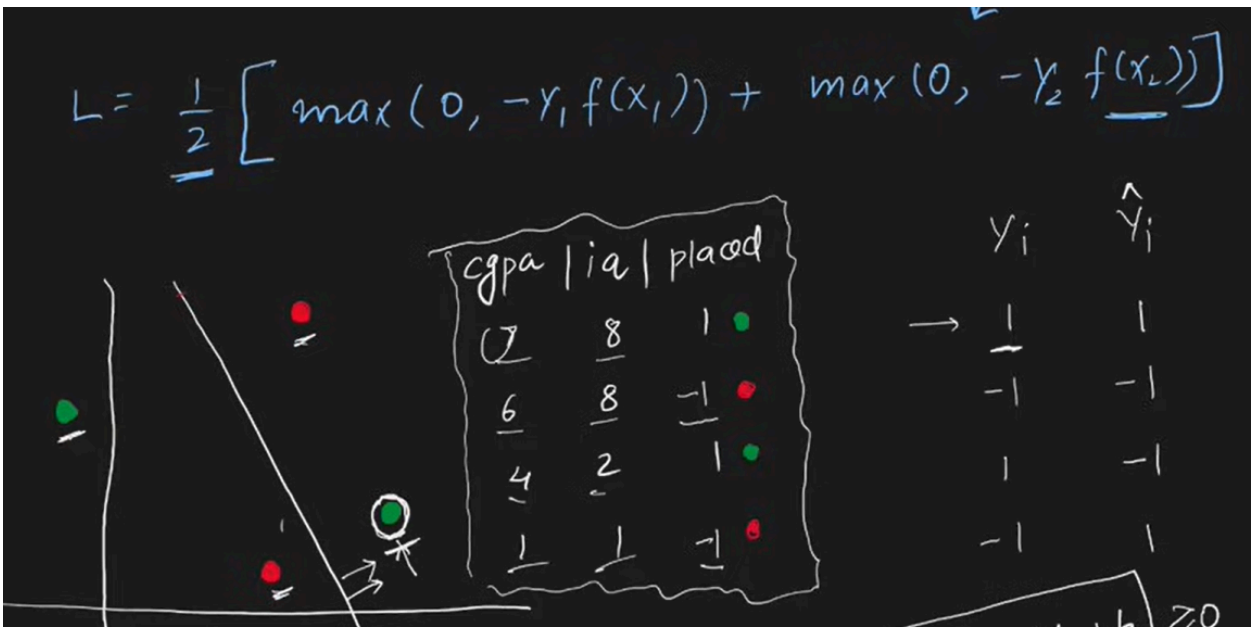


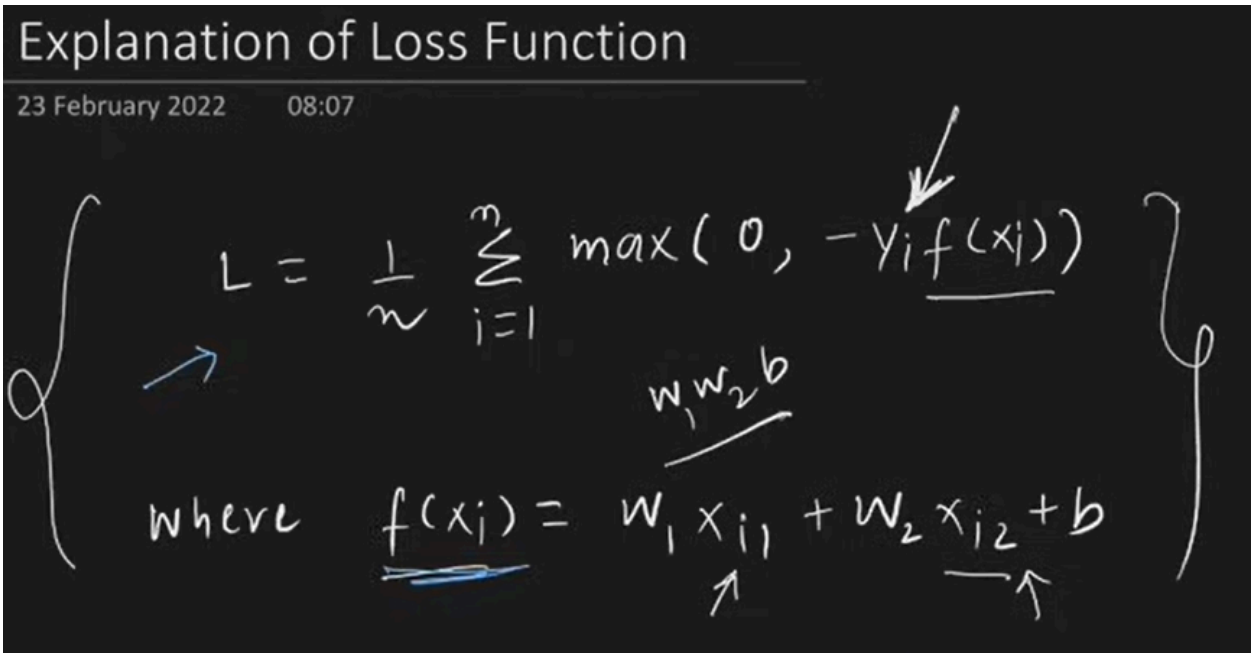

```python
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def activation(self, x):
        return np.where(x >= 0, 1, 0)

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.epochs):
            for i in range(n_samples):
                linear_output = np.dot(X[i], self.weights) +
self.bias
                y_pred = self.activation(linear_output)

                update = self.lr * (y[i] - y_pred)
```

```
                self.weights += update * X[i]
                self.bias += update

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return self.activation(linear_output)


# AND dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])

model = Perceptron(learning_rate=0.1, epochs=10)
model.fit(X, y)

print("Predictions:", model.predict(X))
print("Actual     :", y)

# Visualization
plt.figure()
for i in range(len(X)):
    if y[i] == 0:
        plt.scatter(X[i][0], X[i][1])
    else:
        plt.scatter(X[i][0], X[i][1])

x1 = np.linspace(-0.5, 1.5, 10)
x2 = -(model.weights[0] * x1 + model.bias) / model.weights[1]
plt.plot(x1, x2)

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Decision Boundary")
plt.show()

# XOR test
```

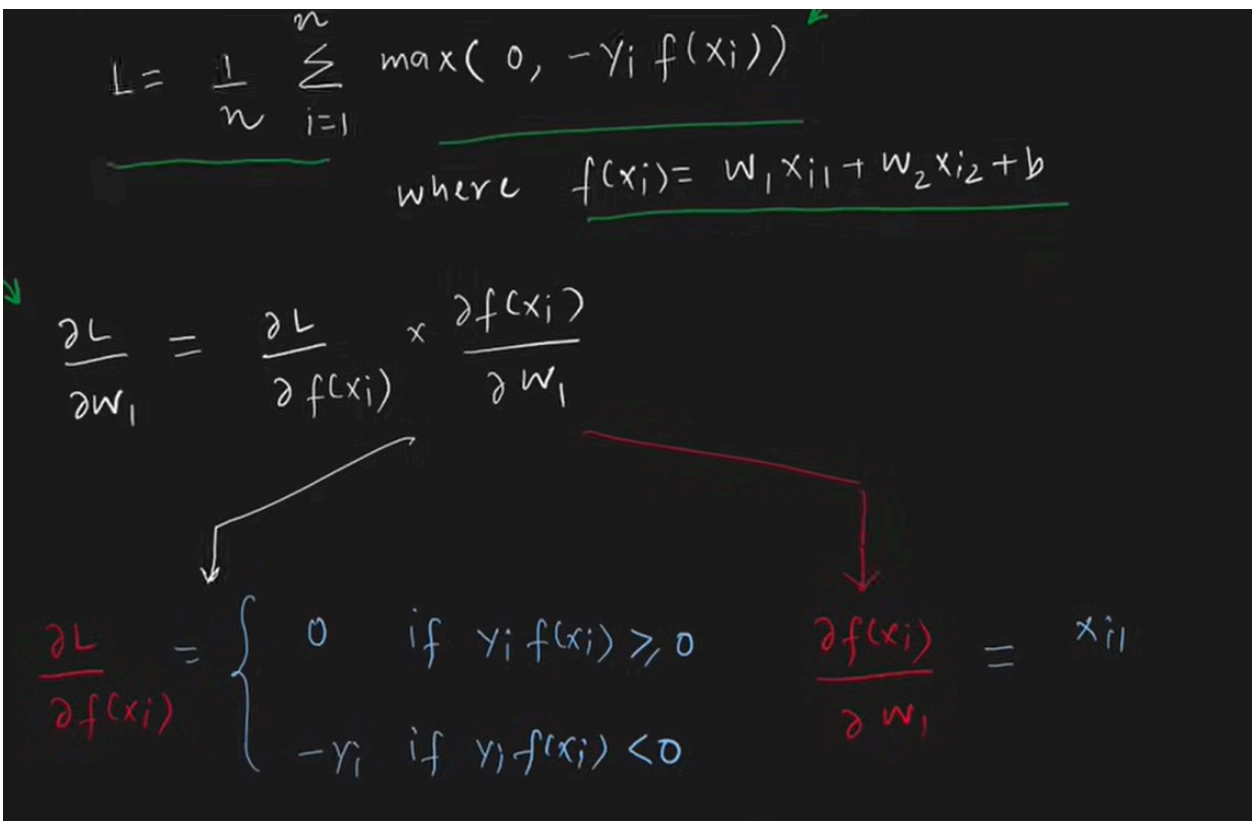
```python
X_xor = np.array([[0,0],[0,1],[1,0],[1,1]])
y_xor = np.array([0,1,1,0])

model_xor = Perceptron(learning_rate=0.1, epochs=10)
model_xor.fit(X_xor, y_xor)

print("XOR Predictions:", model_xor.predict(X_xor))
print("XOR Actual     :", y_xor)
```

Loss Function in perceptron :





Gradient decent :

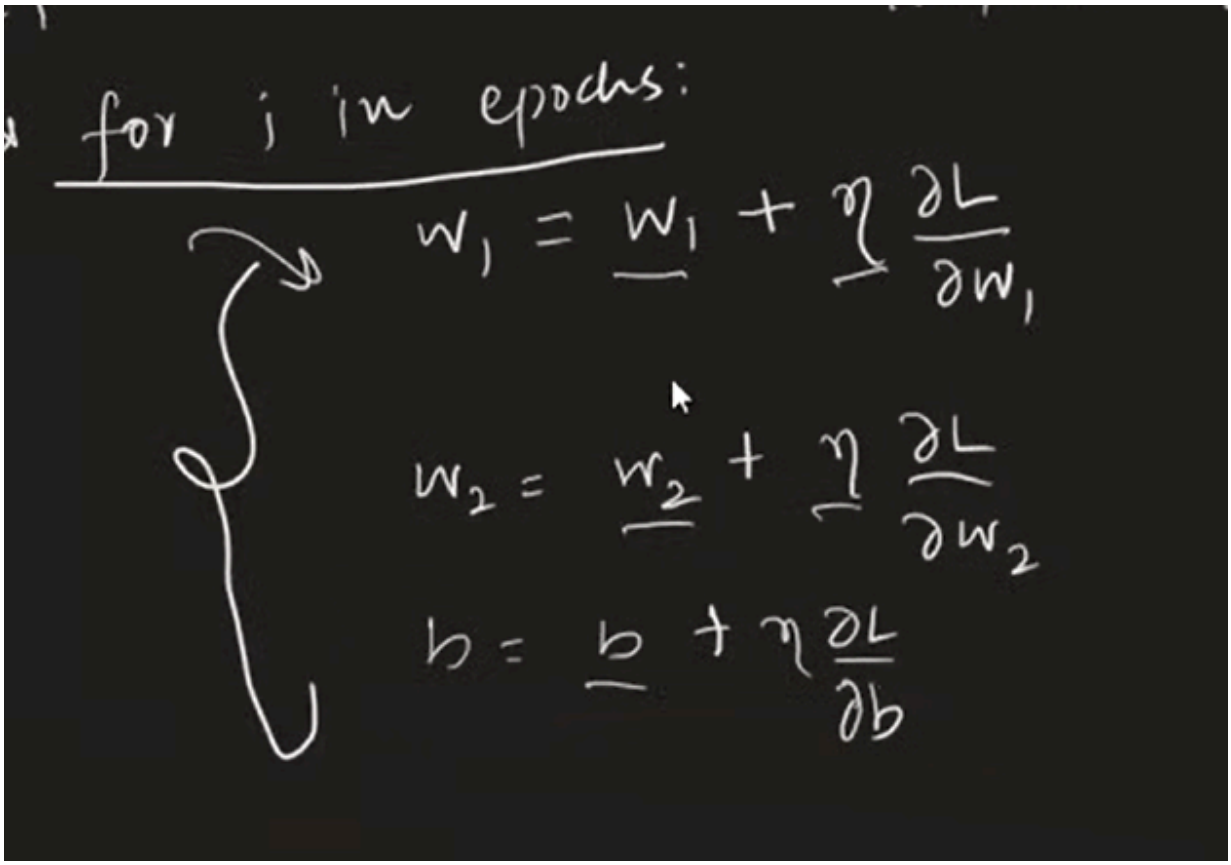


$$\frac{\partial L}{\partial w_1} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{i1} & \text{if } y_i f(x_i) < 0 \end{cases}$$

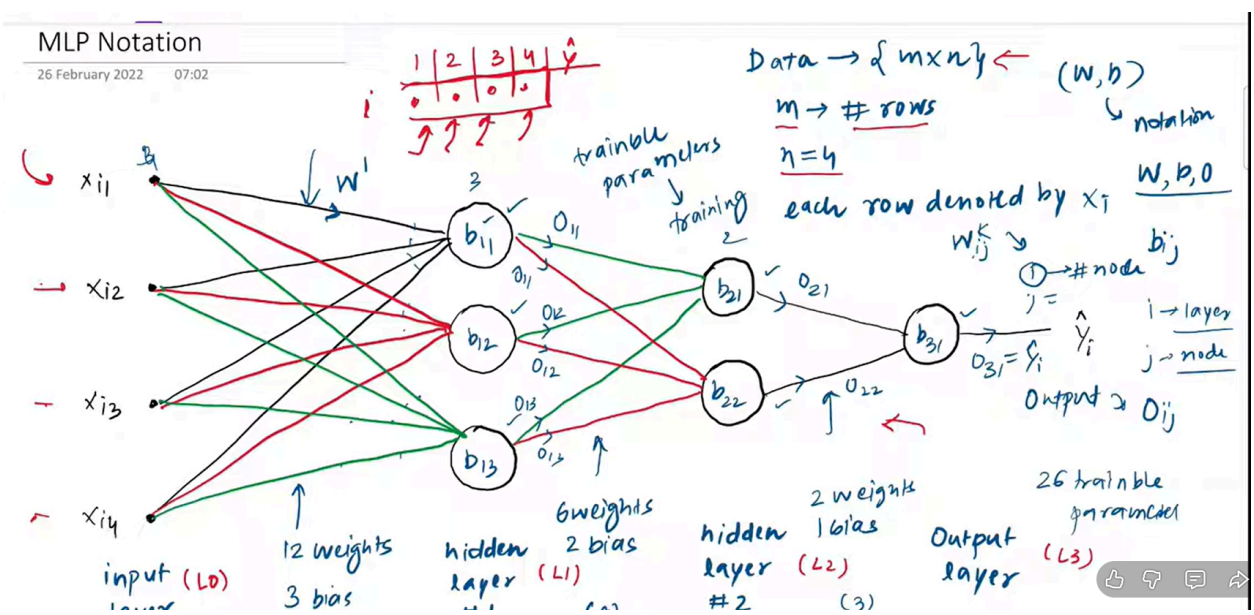
$$\frac{\partial L}{\partial w_2} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i x_{i2} & \text{if } y_i f(x_i) < 0 \end{cases}$$

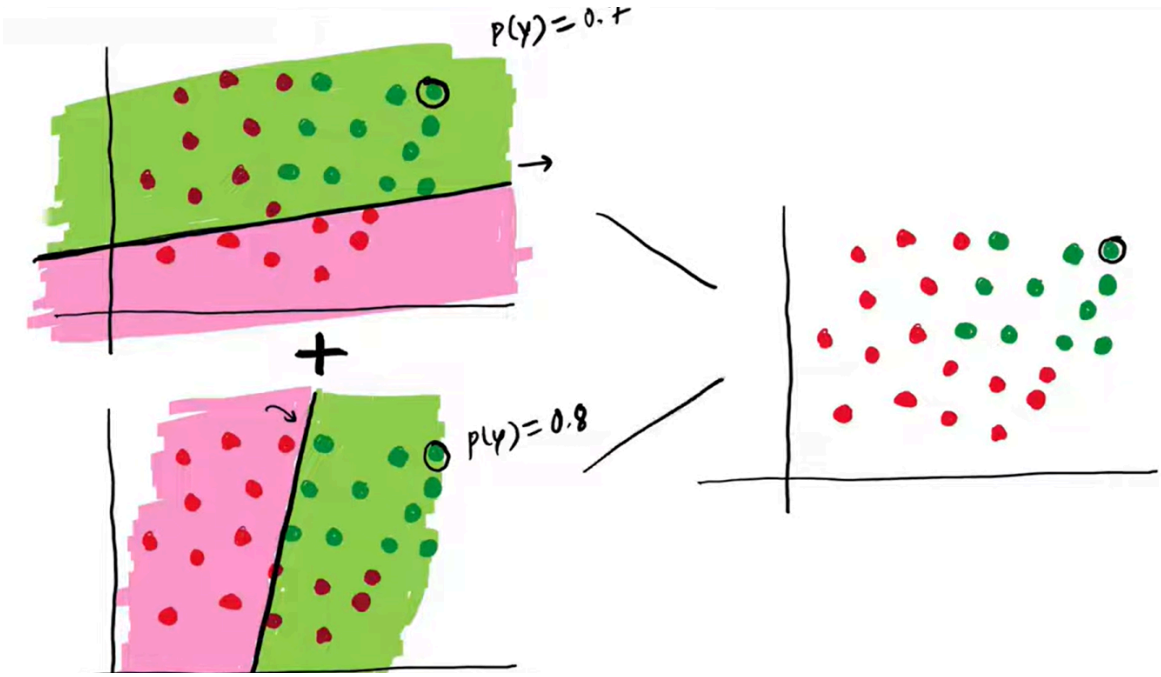
→ code

$$\frac{\partial L}{\partial b} = \begin{cases} 0 & \text{if } y_i f(x_i) \geq 0 \\ -y_i & \text{if } y_i f(x_i) < 0 \end{cases}$$


for i in epochs:
$w_1 = w_1 + \eta \frac{\partial L}{\partial w_1}$
$w_2 = w_2 + \eta \frac{\partial L}{\partial w_2}$
$b = b + \eta \frac{\partial L}{\partial b}$

MLP :


MLP Notation
26 February 2022 07:02
Data → {m×n}
(W, b)
notation
m → # rows
n = 4
each row denoted by x_i
W, b, O
trainable parameters
training
W^k_{ij}
b_{ij}
node
i → layer
j → node
Output → O_{ij}
x_{i1}
x_{i2}
x_{i3}
x_{i4}
W^1
b_{11}
b_{12}
b_{13}
b_{21}
b_{22}
b_{31}
O_{11}
O_{12}
O_{13}
O_{21}
O_{22}
$O_{31} = \hat{y}_i$
$\hat{y}_i$
12 weights
3 bias
6 weights
2 bias
2 weights
1 bias
26 trainable parameters
input (L0)
hidden layer (L1)
hidden layer #2 (L2)
Output layer (L3)


p(y) = 0.4
+
p(y) = 0.8